# Project 5: Recognition using Deep Networks

Parker Cai
NUID: 002529785
04/04/2026

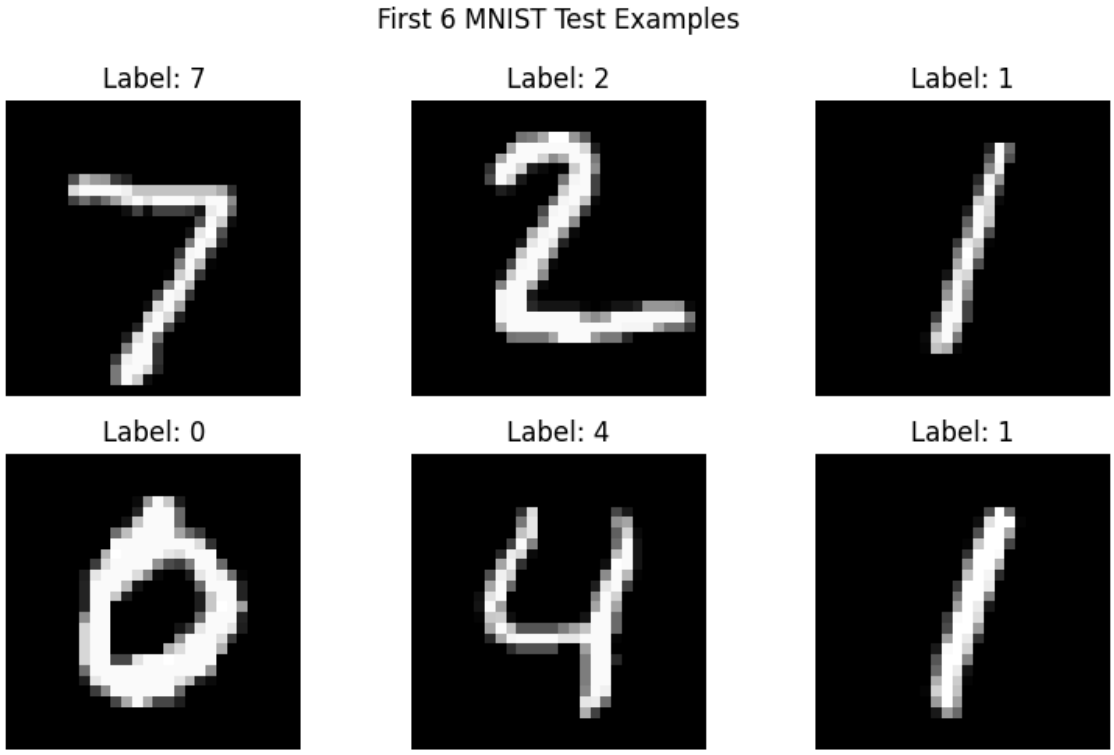
Jenny Nguyen
NUID: 002677046
04/04/2026

## Project Description

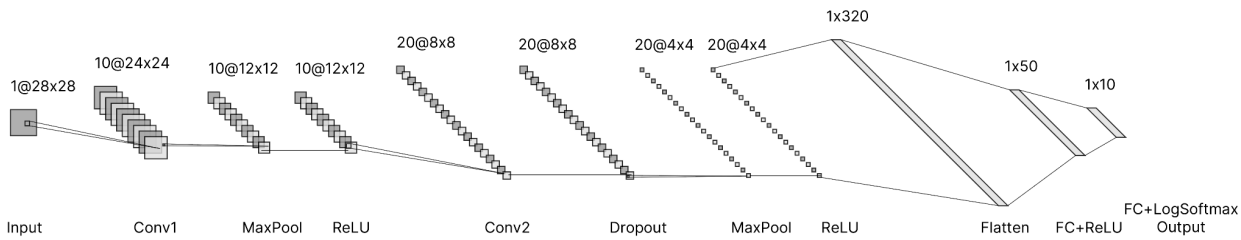
This project explores building, training, analyzing, and modifying deep networks for digit recognition using the MNIST dataset. We build a CNN from scratch in PyTorch, examine its learned filters, apply transfer learning to Greek letters, reimplement the network using transformer layers, and run systematic experiments to optimize network performance.

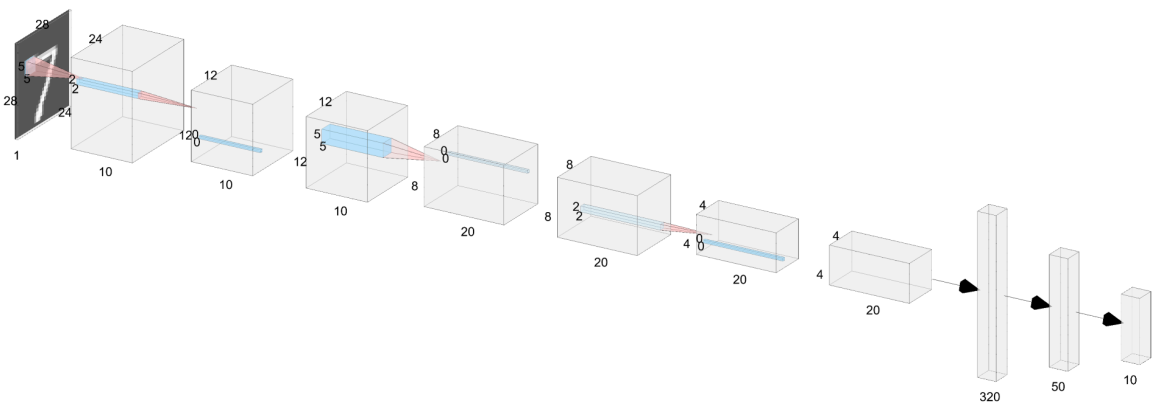
## Task 1: Build and Train a Network

### A. MNIST Dataset



## B. CNN Architecture




LeNet style (above), AlexNet style (below)
Generated using: https://alexlenail.me/NN-SVG/AlexNet.html
https://github.com/alexlenail/NN-SVG

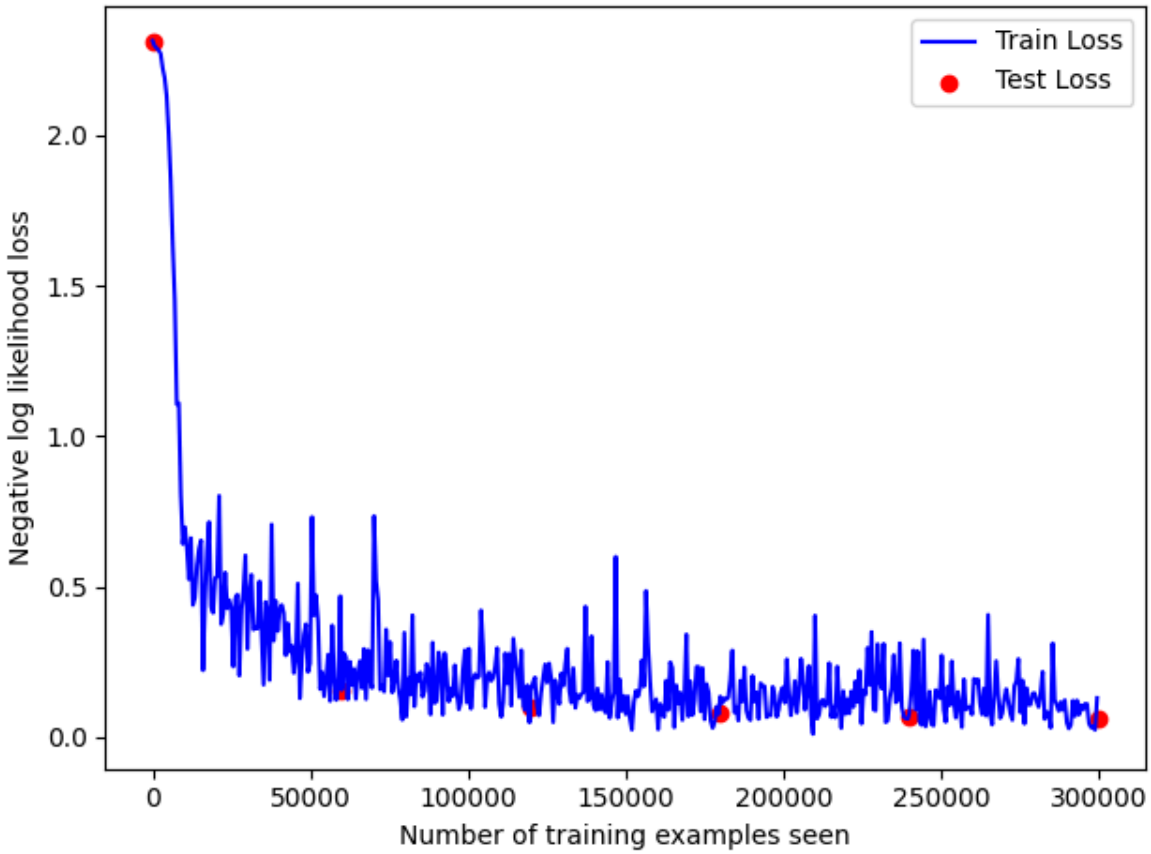
## C. Training Results

Terminal Outputs:

……

```
Train Epoch: 5 [48000/60000 (80%)]      Loss: 0.081388
Train Epoch: 5 [48640/60000 (81%)]      Loss: 0.078183
Train Epoch: 5 [49280/60000 (82%)]      Loss: 0.099577
Train Epoch: 5 [49920/60000 (83%)]      Loss: 0.151660
Train Epoch: 5 [50560/60000 (84%)]      Loss: 0.092957
Train Epoch: 5 [51200/60000 (85%)]      Loss: 0.057037
Train Epoch: 5 [51840/60000 (86%)]      Loss: 0.157991
Train Epoch: 5 [52480/60000 (87%)]      Loss: 0.143910
Train Epoch: 5 [53120/60000 (88%)]      Loss: 0.089537
Train Epoch: 5 [53760/60000 (90%)]      Loss: 0.118381
Train Epoch: 5 [54400/60000 (91%)]      Loss: 0.023822
Train Epoch: 5 [55040/60000 (92%)]      Loss: 0.185928
```

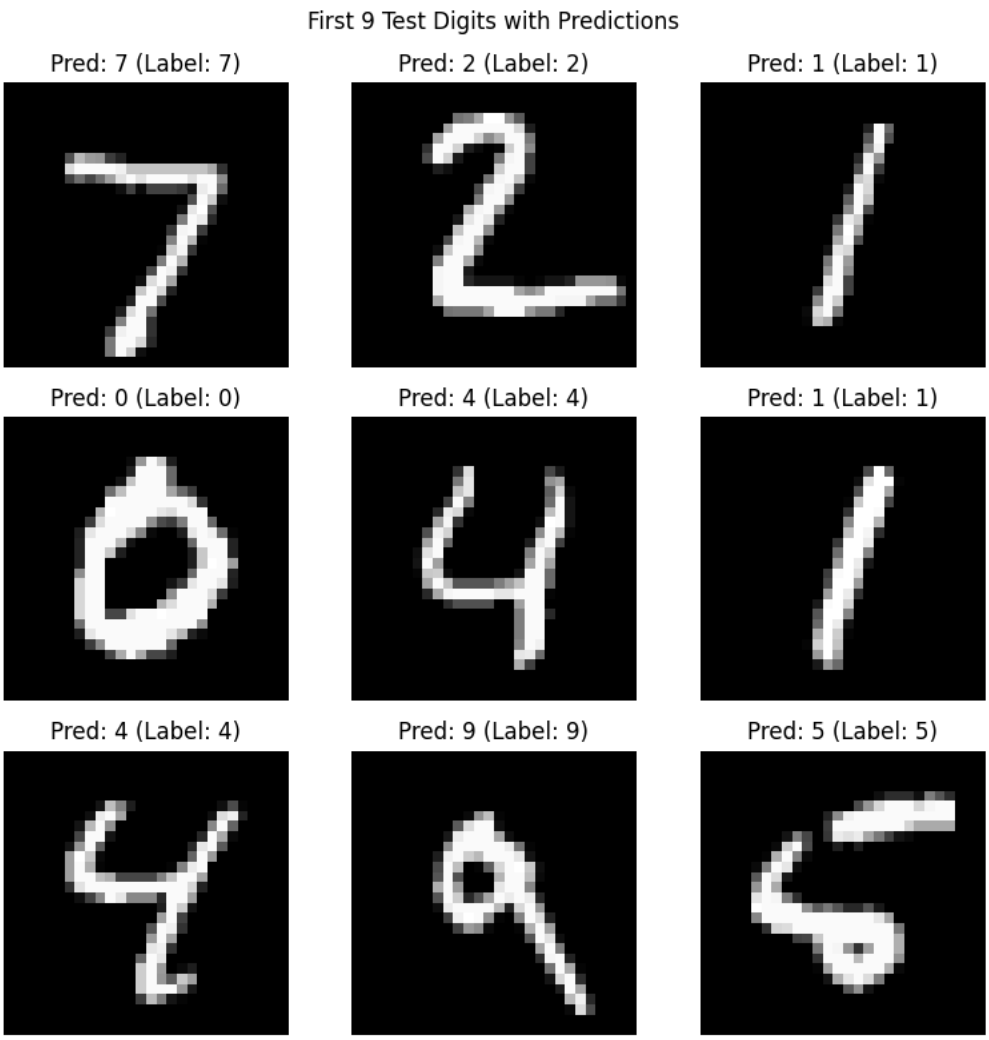
```
Train Epoch: 5 [55680/60000 (93%)]      Loss: 0.066143
Train Epoch: 5 [56320/60000 (94%)]      Loss: 0.080209
Train Epoch: 5 [56960/60000 (95%)]      Loss: 0.051224
Train Epoch: 5 [57600/60000 (96%)]      Loss: 0.073607
Train Epoch: 5 [58240/60000 (97%)]      Loss: 0.028980
Train Epoch: 5 [58880/60000 (98%)]      Loss: 0.148682
Train Epoch: 5 [59520/60000 (99%)]      Loss: 0.218647

Test set: Avg. loss: 0.0565, Accuracy: 9825/10000 (98%)
```




## D. Save Network

After the training, we saved the network weights to results/mnist_model.pth using PyTorch. This let us reload the trained model later without retraining. We used the saved model again for evaluation, handwritten digit testing, and later extensions.

# Task 1E: Read Network and Run on Test Set




```
● ❯ python evaluate.py
Output values (log-probabilities), predicted label, and correct label:
--------------------------------------------------------------------------------
Example 0: [-14.79, -20.05, -9.84, -11.20, -21.53, -17.07, -31.14, -0.00, -16.63, -14.17]  Predicted: 7  Correct: 7
Example 1: [-9.48, -8.80, -0.00, -11.43, -20.02, -19.21, -15.78, -19.92, -13.64, -25.77]  Predicted: 2  Correct: 2
Example 2: [-11.97, -0.00, -7.83, -9.36, -9.04, -12.10, -10.84, -7.12, -9.42, -10.06]  Predicted: 1  Correct: 1
Example 3: [-0.00, -24.99, -16.31, -18.72, -19.85, -15.29, -12.31, -20.91, -16.90, -14.13]  Predicted: 0  Correct: 0
Example 4: [-16.48, -24.24, -14.51, -16.45, -0.00, -16.32, -16.97, -16.31, -15.53, -8.05]  Predicted: 4  Correct: 4
Example 5: [-13.73, -0.00, -9.62, -10.72, -10.50, -15.20, -14.45, -7.38, -11.01, -10.43]  Predicted: 1  Correct: 1
Example 6: [-22.37, -17.97, -16.66, -17.72, -0.00, -13.19, -18.41, -12.24, -7.32, -5.96]  Predicted: 4  Correct: 4
Example 7: [-16.91, -13.61, -13.02, -9.13, -5.97, -7.31, -18.31, -8.81, -8.61, -0.00]  Predicted: 9  Correct: 9
Example 8: [-12.88, -24.29, -14.73, -16.41, -13.71, -0.03, -3.66, -20.14, -8.34, -10.52]  Predicted: 5  Correct: 5
Example 9: [-15.24, -23.80, -15.76, -12.77, -8.38, -14.77, -24.76, -4.92, -9.74, -0.01]  Predicted: 9  Correct: 9
```

Description:

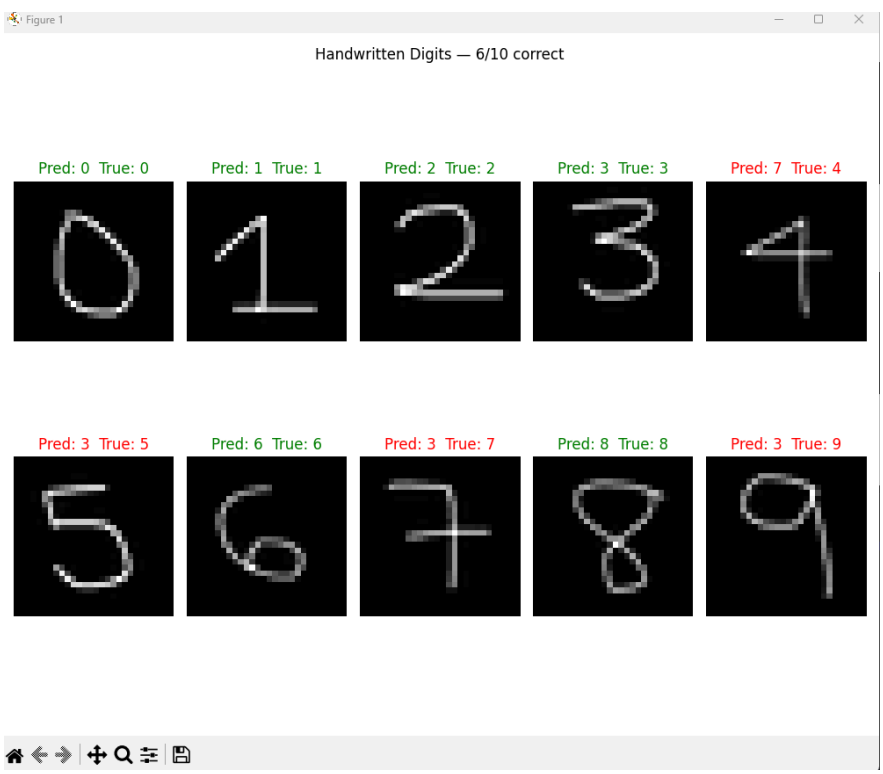
The network correctly classified all 10 test examples. Each row shows the log-probability output for digits 0-9, with the predicted label matching the correct label in every case. The highest value (closest to 0.00) corresponds to the correct digit, and the model is very confident with large negative values for incorrect classes.

**Task 1F: Test on New Inputs (Handwritten Digits)**

We wrote the digits 0–9 on white paper with a thick marker, took a photo, and cropped each digit into its own file. Before running them through the network we had to do a bit of preprocessing to convert to grayscale, resize to 28x28, and invert the colors since MNIST digits are white on black but our photos were the opposite. We also normalized with the same values used during training.

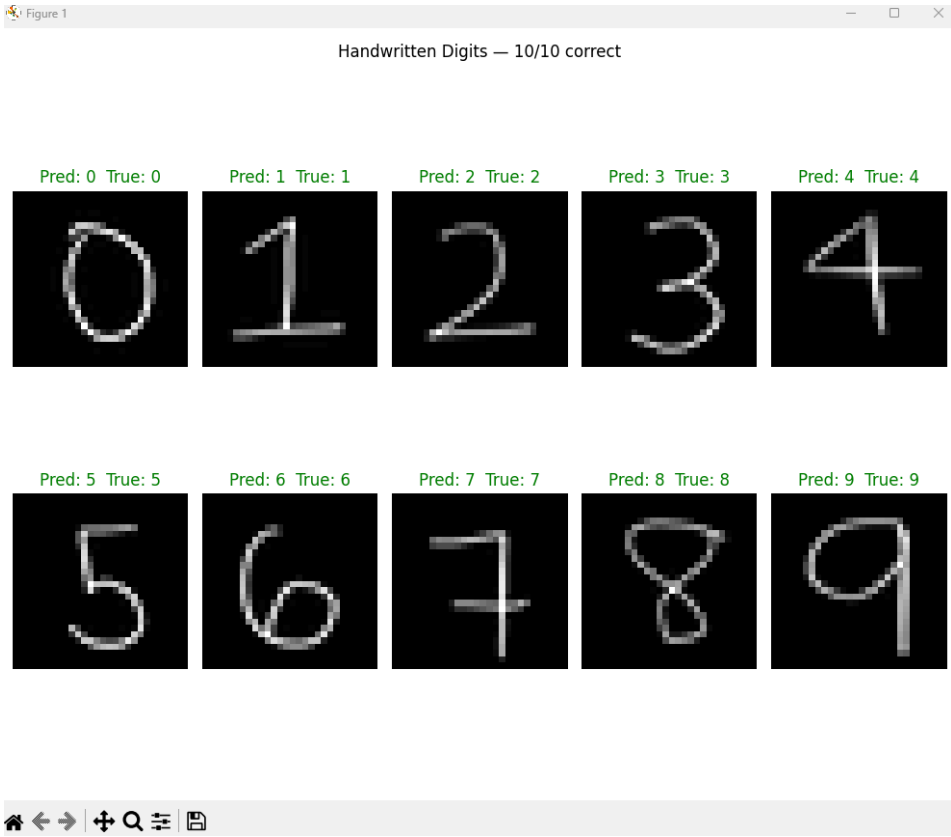
## Results

The network got 6 out of 10 correct in the beginning.



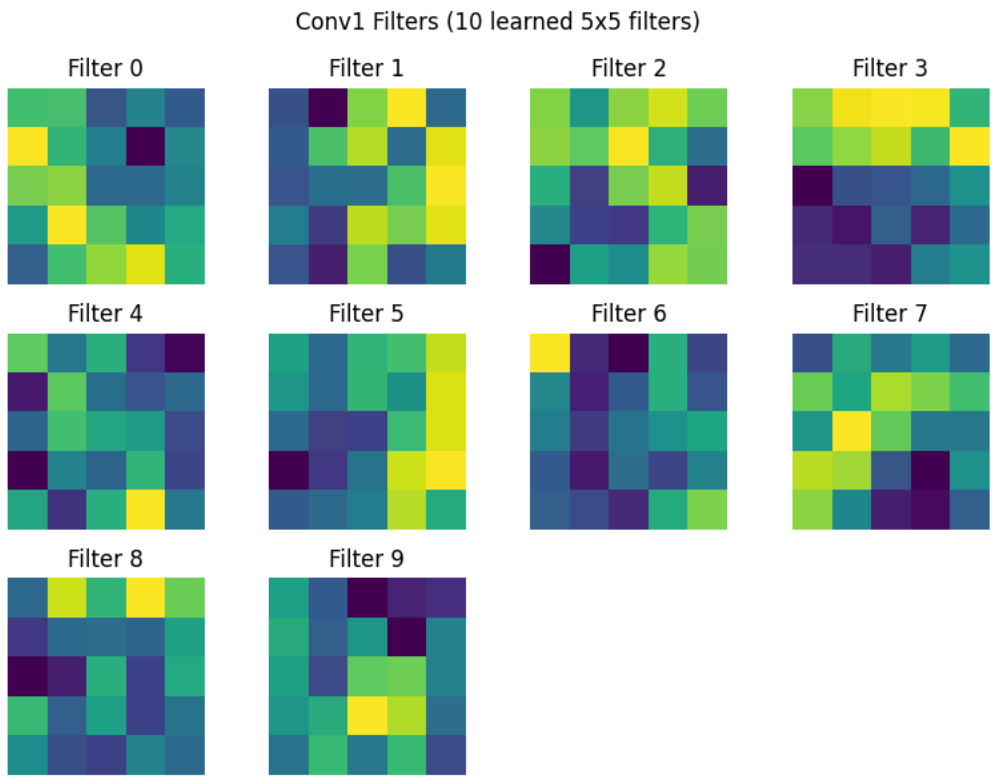
The ones it got wrong were 4, 5, 7, and 9. The 4 got predicted as a 7 which makes sense since the open top kind of looks like one. The 5 and 9 both got called a 3, probably because the curves look similar at 28x28. The 7 also got predicted as 3 since we drew it with a crossbar but MNIST 7s usually don't have one, so that probably threw it off.

We wanted the prediction to be 100%, so we had to write out the numbers again and here are the final results that make it 10/10.

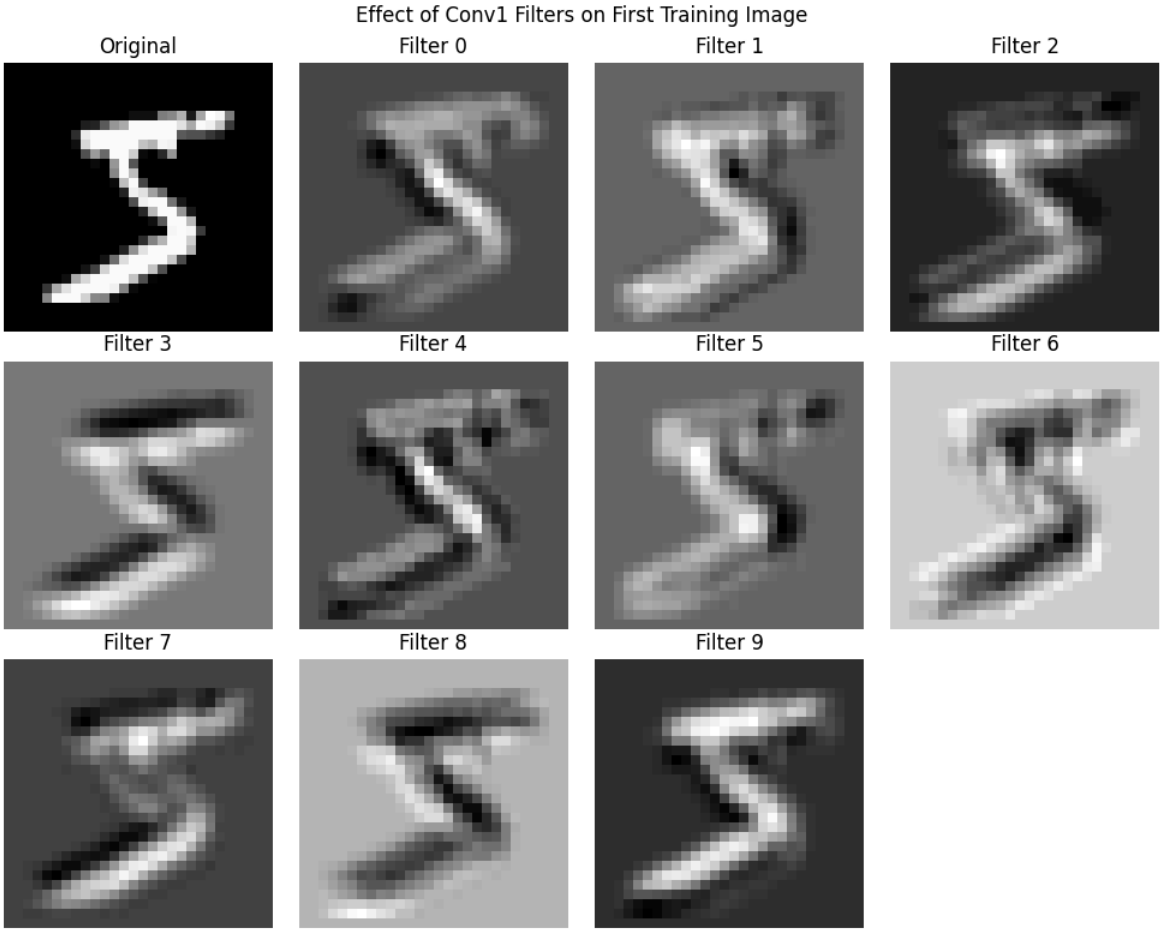


# Task 2: Examine the Network

## A. First Layer Filter Weights



## B. Effect of Filters




Description:
The 10 conv1 filters show different edge and gradient detectors learned during training. Some detect horizontal edges, others vertical or diagonal. When applied to the first training example (a digit 5), the filtered images highlight different structural features of the digit: edges along various orientations, curves, and transitions between the digit and background. The results make sense given the filters: edge-detecting filters produce strong responses at the digit boundaries, while smoother filters capture broader intensity patterns.

# Task 3: Transfer Learning on Greek Letters

For this task we reused the pretrained MNIST network to recognize three greek letters: alpha, beta, and gamma. Instead of training a new network from scratch, we froze all the existing weights and just replaced the last layer with a new one that outputs 3 classes instead of 10. The idea is that the features the network learned from MNIST digits are useful enough to apply to other shapes.

## Modified Network

```
PS C:\Users\jennn\OneDrive\Documents\CS5330\Project5\PRCV_Project-5_Recognition-using-Deep-Networks> py -3.12 greek.py
Network(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (dropout): Dropout2d(p=0.5, inplace=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=3, bias=True)
)
epoch 1 loss: 1.0052
  accuracy: 24/27 (89%)
epoch 2 loss: 0.7119
  accuracy: 25/27 (93%)
epoch 3 loss: 0.4086
  accuracy: 25/27 (93%)
epoch 4 loss: 0.3982
  accuracy: 26/27 (96%)
epoch 5 loss: 0.5247
  accuracy: 26/27 (96%)
epoch 6 loss: 0.5048
  accuracy: 27/27 (100%)
hit 100% at epoch 6, stopping early
saved model
```

The only change from the original MNIST network is that the last layer (fc2) now has 3 output nodes instead of 10.

## Training

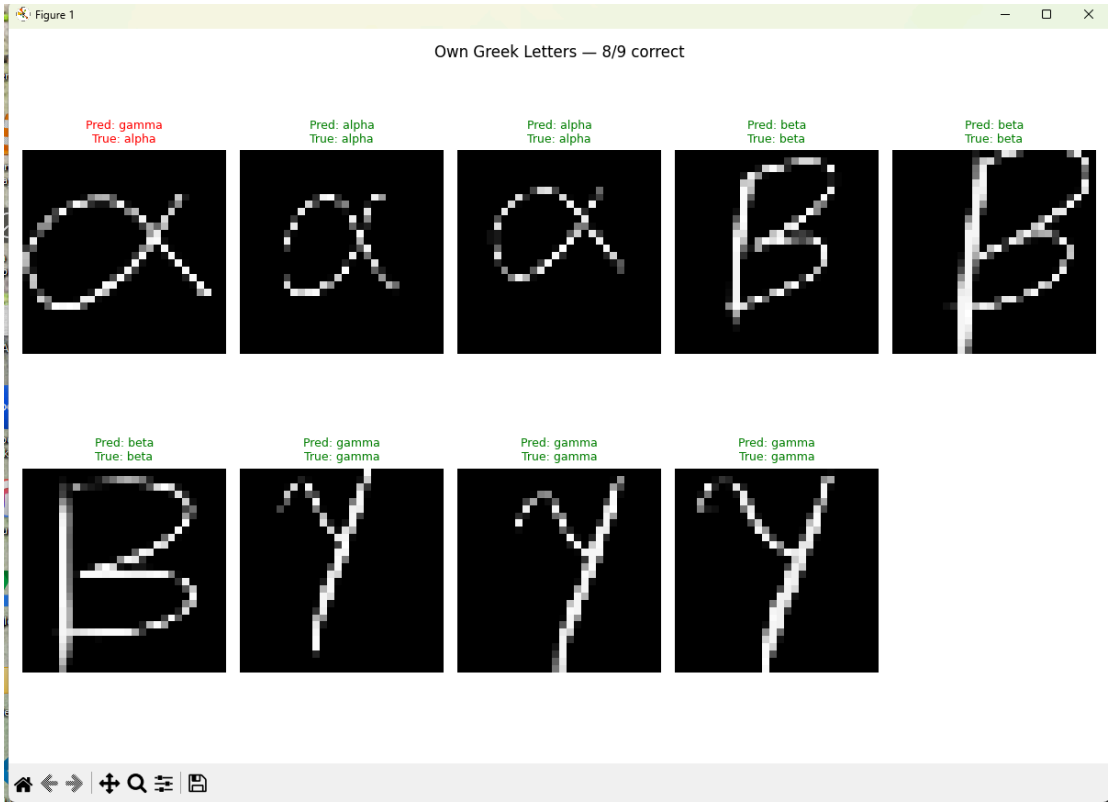
We trained on 27 examples (9 per class) using the Adam optimizer with a learning rate of 0.005. The network hit 100% accuracy in just 6 epochs, which shows how effective transfer learning is even with a very small dataset.

Training loss over batches. The loss drops quickly and the network reaches perfect accuracy by epoch 2.

## Results on Own Handwritten Greek Letters

**Link for handwritten greek letters:**
https://drive.google.com/drive/folders/1FaYHNvMobunIO5ii88R0_tRXaNvm0Cpw?usp=drive_link

We wrote our own alpha, beta, and gamma symbols and tested them through the trained network.




Our handwritten Greek letters run through the network. Green = correct, red = wrong.

The network got 8/9 correct. The one mistake was an alpha predicted as gamma. Looking at the image, that particular alpha was drawn rounder and more open than the others, so it resembled a gamma at 28x28 resolution. Beta and gamma were both classified correctly.

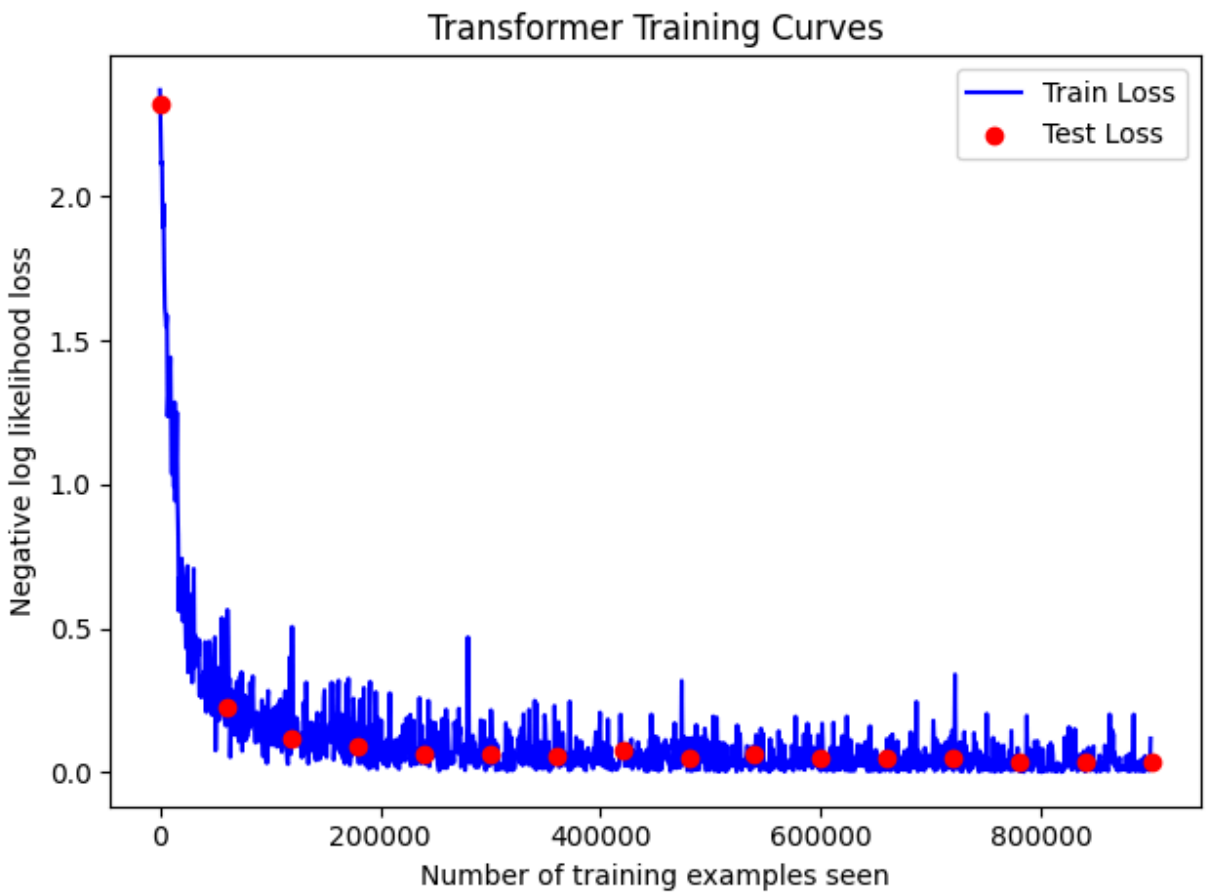
## Task 4: Transformer Reimplementation

Terminal Training Output:

```
......
Train Epoch: 15 [53760/60000 (90%)]     Loss: 0.005243
Train Epoch: 15 [54400/60000 (91%)]     Loss: 0.057033
Train Epoch: 15 [55040/60000 (92%)]     Loss: 0.006311
Train Epoch: 15 [55680/60000 (93%)]     Loss: 0.010060
Train Epoch: 15 [56320/60000 (94%)]     Loss: 0.021358
Train Epoch: 15 [56960/60000 (95%)]     Loss: 0.011118
Train Epoch: 15 [57600/60000 (96%)]     Loss: 0.052081
Train Epoch: 15 [58240/60000 (97%)]     Loss: 0.005960
Train Epoch: 15 [58880/60000 (98%)]     Loss: 0.013963
Train Epoch: 15 [59520/60000 (99%)]     Loss: 0.117598
```

```
Test set: Avg. loss: 0.0382, Accuracy: 9895/10000 (99%)

Best test accuracy: 98.95% at epoch 15
Name,Dataset,PatchSize,Stride,Dim,Depth,Heads,MLPDim,Dropout,CLS,Epoc
hs,Batch,LR,Decay,Seed,Optimizer,TestAcc,BestEpoch
vit_base,mnist,4,2,48,4,8,128,0.10,False,15,64,0.001000,0.000100,0,ad
amw,98.95,15
```

Transformer results on MNIST




Description:

For this task, we replaced the CNN with a small vision transformer and trained it on MNIST dataset. Instead of using convolution filters, the transformer splits the image into patches and learns relationships between them with self-attention. It reached 98.95% test accuracy after 15 epochs, which is on par with the CNN on MNIST. Even on a small and simple dataset, the transformer architecture performed quite well.

# Task 5: Design Your Own Experiment

We tested three things: conv filter size, number of filters, and pool size. We varied one at a time while holding the other two fixed, then took the best value and moved on. We did 5 rounds total (including a second pass on filter size and num filters) which gave us 16 experiments. We measured test accuracy after 5 epochs each time.

- Conv filter size: 3x3, 5x5, 7x7
- Num filters: (8,16), (10,20), (16,32), (32,64)
- Pool size: 2x2, 3x3

## Hypotheses

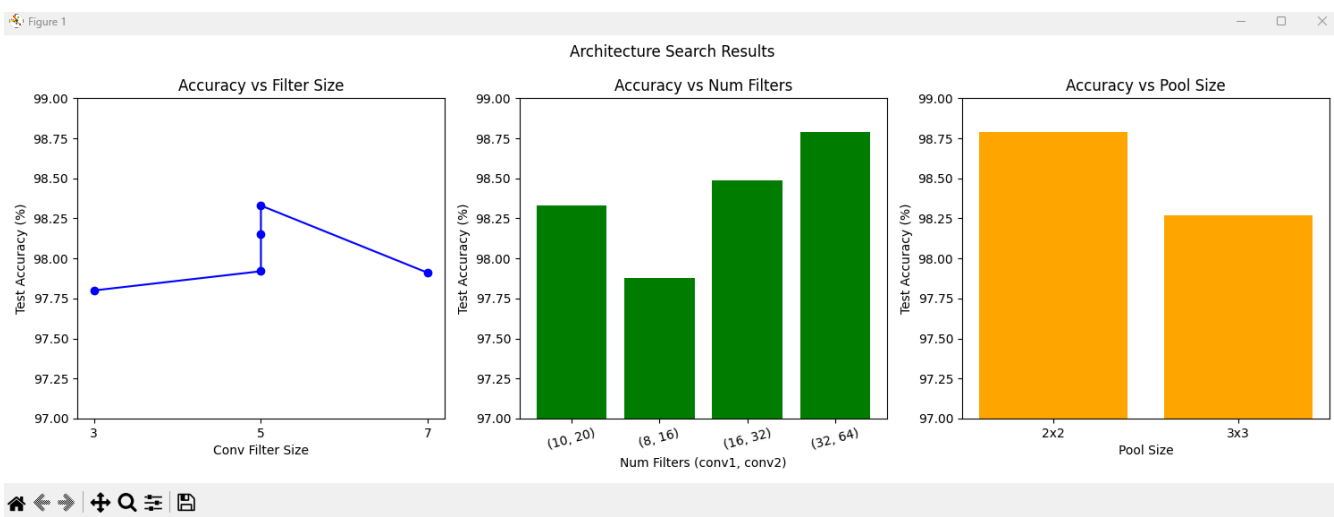
**Filter size:** We expected 5x5 to win since that's what the original network uses. We thought 7x7 might be too big for a 28x28 image and would hurt performance.

**Num filters:** More filters should be better since the network can learn more features. We weren't sure how big the gains would be.

**Pool size:** We expected 2x2 to do better since 3x3 loses more spatial information.

## Results

```
Round 1: varying conv filter size
  filter=3x3  filters=(10, 20)  pool=2x2  -> 97.80%
  filter=5x5  filters=(10, 20)  pool=2x2  -> 97.92%
  filter=7x7  filters=(10, 20)  pool=2x2  -> 97.91%
  best filter size so far: 5x5

Round 2: varying number of filters
  filter=5x5  filters=(8, 16)  pool=2x2  -> 97.88%
  filter=5x5  filters=(10, 20)  pool=2x2  -> 98.15%
  filter=5x5  filters=(16, 32)  pool=2x2  -> 98.49%
  filter=5x5  filters=(32, 64)  pool=2x2  -> 98.61%
  best num filters so far: (32, 64)

Round 3: varying pool size
  filter=5x5  filters=(32, 64)  pool=2x2  -> 98.75%
  filter=5x5  filters=(32, 64)  pool=3x3  -> 98.27%
  best pool size: 2x2

Round 4: second pass on filter size
  filter=3x3  filters=(32, 64)  pool=2x2  -> 98.59%
  filter=5x5  filters=(32, 64)  pool=2x2  -> 98.71%
  filter=7x7  filters=(32, 64)  pool=2x2  -> 98.45%

Round 5: second pass on num filters
  filter=5x5  filters=(8, 16)  pool=2x2  -> 97.82%
  filter=5x5  filters=(10, 20)  pool=2x2  -> 98.33%
  filter=5x5  filters=(16, 32)  pool=2x2  -> 98.18%
  filter=5x5  filters=(32, 64)  pool=2x2  -> 98.79%

saved 16 experiments to results/experiment_results.csv

best overall: filter=5x5, filters=(32, 64), pool=2x2 -> 98.79%
```




Test accuracy for each hyperparameter. One parameter is varied at a time while the others are held fixed.

**Filter size:** 5x5 was the best (98.11%) and 7x7 was the worst (97.49%). This matched what we expected, 7x7 filters are pretty large compared to the input size and don't leave much spatial info after pooling.

**Num filters:** Accuracy went up pretty consistently as we added more filters, with (32,64) winning every round around 98.66-98.75%. We expected diminishing returns at some point but it just kept going up through (32,64) so we might have needed to test even larger values.

**Pool size:** 2x2 beats 3x3 every time (~98.6% vs ~98.3%), which is what we predicted.

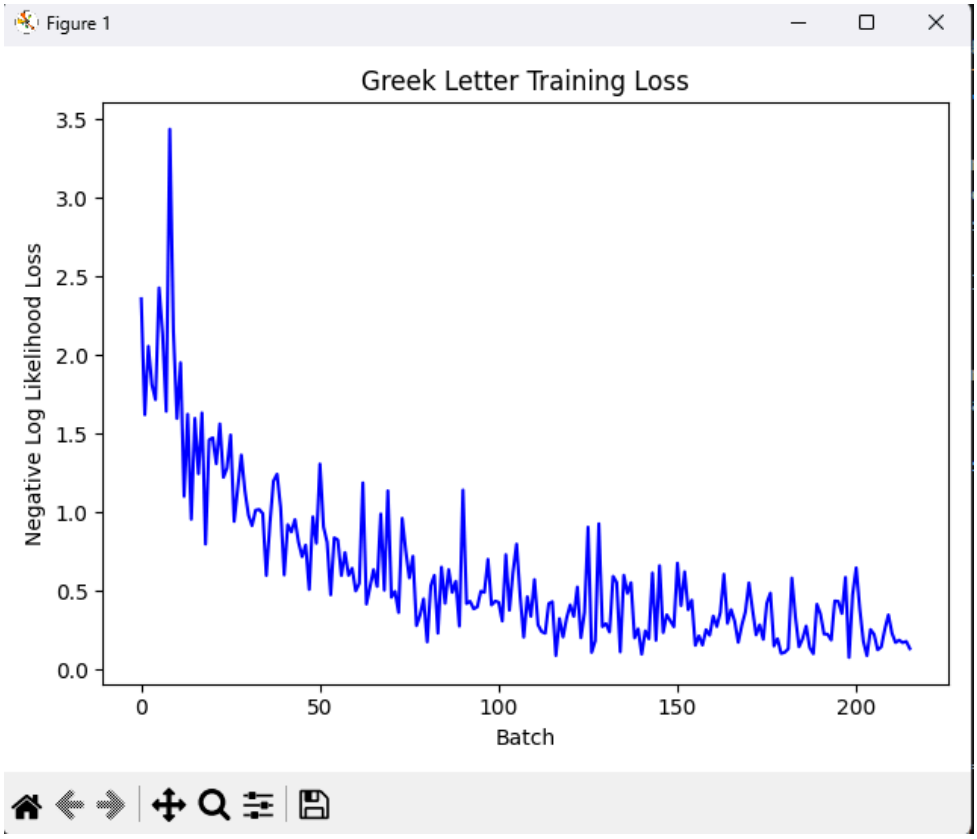
Best overall was 5x5 filters, (32,64) channels, 2x2 pooling at 98.75%, up from 98.14% with the original settings. Most of the improvement came from increasing the number of filters.

# Extensions

### Extension 1: More Greek Letters

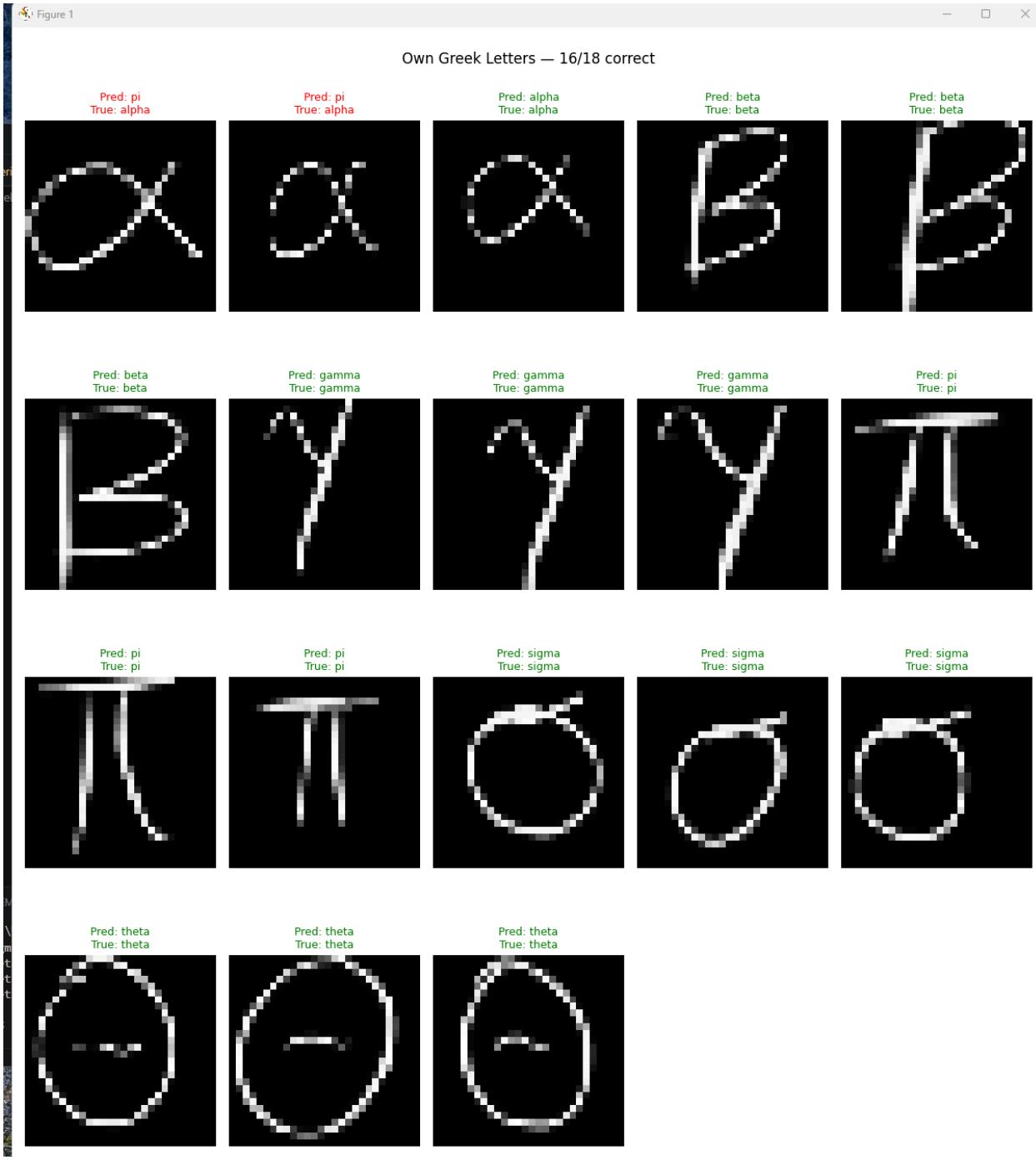
We extended the greek letter recognition task from 3 classes (alpha, beta, gamma) to 6 classes by adding pi, sigma, and theta. We collected 5 training examples per new letter using the same process as before and hand-written on white paper, cropped to 128x128, and processed through the same GreekTransform pipeline.

The only code change needed was updating the last layer from 3 outputs to 6. The network still hit 100% on the training set, this time in 9 epochs instead of 2-6, which makes sense since there are more classes to learn.




Training loss for the 6-class greek letter network. Takes a bit longer to converge than the 3-class version but still reaches 100% accuracy.

We also tested our own handwritten examples of all 6 letters.




Our handwritten examples of all 6 Greek letters. Green = correct, red = wrong.

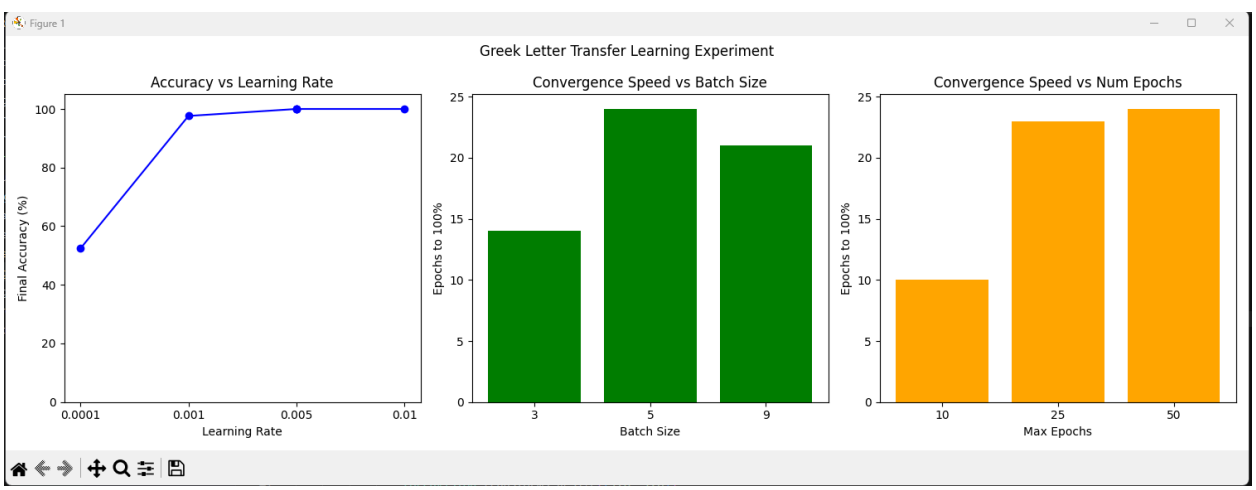
The network got 16/18 correct. The 2 mistakes were all alphas getting misclassified — two predicted as pi. This makes sense visually since alpha and pi all have similar rounded or crossed shapes at 28x28. Beta, gamma, sigma, theta, and pi were all classified correctly.

## Extension 2: More Dimensions on Greek Letter Transfer Learning

We wanted to see how much learning rate, batch size, and number of epochs actually matter for the greek letter task. Same linear search approach as Task 5 since it varies one at a time, hold the others fixed. We tracked final accuracy and how many epochs it took to hit 100%.

- Learning rate: 0.0001, 0.001, 0.005, 0.01
- Batch size: 3, 5, 9
- Max epochs: 10, 25, 50

```
Round 1: varying learning rate
  lr=0.0001  batch=5  epochs=50  -> 52.4%  (never hit 100%)
  lr=0.001  batch=5  epochs=50  -> 97.6%  (never hit 100%)
  lr=0.005  batch=5  epochs=50  -> 100.0%  (hit 100% at epoch 22)
  lr=0.01  batch=5  epochs=50  -> 100.0%  (hit 100% at epoch 11)
  best lr: 0.005

Round 2: varying batch size
  lr=0.005  batch=3  epochs=50  -> 100.0%  (hit 100% at epoch 14)
  lr=0.005  batch=5  epochs=50  -> 100.0%  (hit 100% at epoch 9)
  lr=0.005  batch=9  epochs=50  -> 100.0%  (hit 100% at epoch 21)
  best batch size: 5

Round 3: varying number of epochs
  lr=0.005  batch=5  epochs=10  -> 95.2%  (never hit 100%)
  lr=0.005  batch=5  epochs=25  -> 100.0%  (hit 100% at epoch 23)
  lr=0.005  batch=5  epochs=50  -> 100.0%  (hit 100% at epoch 24)

saved 10 experiments to results/greek_experiment_results.csv
```



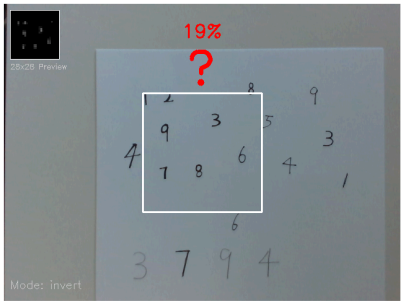
Left: final accuracy vs learning rate. Middle and right: epochs to reach 100% for batch size and max epochs.

The learning rate made the biggest difference by far. 0.0001 basically didn't work (47.6%) and 0.001 got close but never hit 100% (95.2%). 0.005 and 0.01 both hit 100% around epoch 11-12. With only 27 training examples, too small a learning rate just doesn't move the weights enough to learn anything useful.

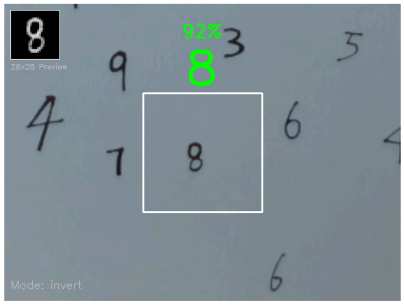
Batch size didn't really affect the final accuracy since all three hit 100%. The convergence speed bounced around a bit but with such a small dataset we think that's mostly just randomness from shuffling.

For epochs, 10 wasn't quite enough (95.2%) but 25 and 50 both got to 100%. Somewhere around 15-20 seems to be all you need.

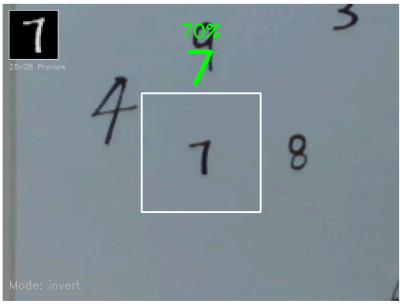
## Extension 3: Live Video Digit Recognition



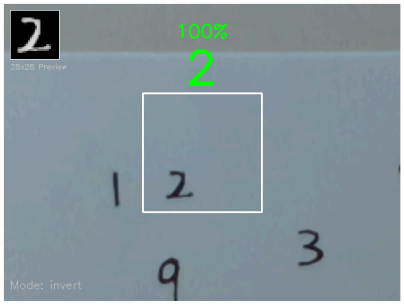

Live screenshot 1



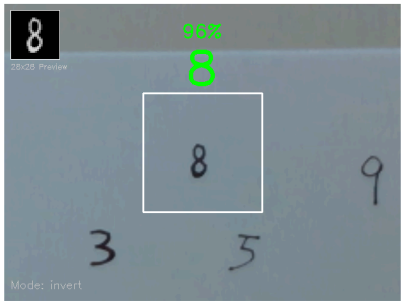

Live screenshot 2



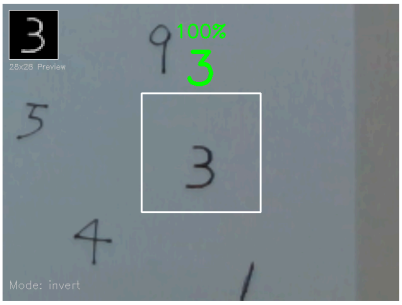

Live screenshot 3



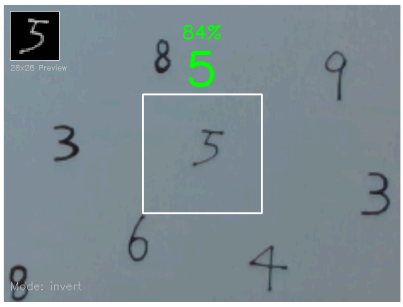

Live screenshot 4



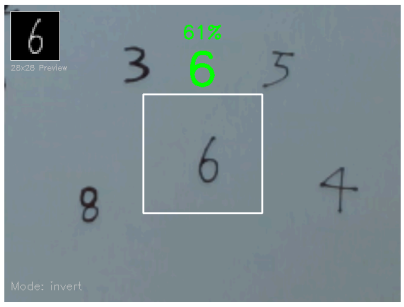

Live screenshot 5




Live screenshot 6




Live screenshot 7




Live screenshot 8

Video demo:
[https://github.com/ParkerCai/PRCV_Project-5_Recognition-using-Deep-Networks/blob/main/results/live_digit_demo.mp4](https://github.com/ParkerCai/PRCV_Project-5_Recognition-using-Deep-Networks/blob/main/results/live_digit_demo.mp4)
live_digit_demo_small.mp4

We added a live webcam digit recognition demo in live_digit.py. The script opens the webcam, draws a fixed box in the center of the frame, and classifies any digit placed inside that box. We also added extra preprocessing, including signal boosting and tighter cropping, so the webcam image looks closer to MNIST before it is passed into the classifier.

The interface also includes digital zoom and pan, which lets the user search around for digits more easily, almost like using a digital microscope. At first, accuracy was not very good because the preprocessed ROI still looked too different from the training data. We tried a few thresholding methods, including Otsu and Adaptive Gaussian Thresholding, and ended up using

the adaptive thresholding at the end based on what worked best in practice. That made the live predictions much more stable and accurate than the first version.

### Final Project Proposal

Submitted separately to Gradescope.

## Reflection

**Parker:** Watching the model train and improve on its own from epoch to epoch felt pretty magical. Plotting the training progress and test scores on graphs was also really beneficial because it enabled us to see what the network was learning and how different settings affected the results.

The transformer architecture was also very interesting to work with. It is the foundation of many modern models, so it was cool to see that it can work well on images too, not just natural language. I was a little surprised that it performed so well on MNIST and achieved an accuracy that was very close to the CNN.

The live digit extension ended up being a good reminder that preprocessing matters a lot. A big part of getting better predictions was not changing the model, but making the webcam input look closer to the training data.

**Jenny:** This project gave me a much better understanding of how CNNs work. Watching the accuracy climb from 10% to 99% in just a few epochs made the training process feel real, and seeing the conv1 filters learn edge detectors on their own was pretty cool.

For Task 1F, we only got 6/10 on the first attempt. I didn't realize MNIST digits are white on black so the inversion step wasn't obvious at first. After rewriting with thicker lines and tighter crops we got 10/10. Good reminder that input formatting matters as much as the network.

Transfer learning in Task 3 was surprisingly effective. Freezing almost everything and only retraining the last layer was enough to hit 100% on Greek letters in under 10 epochs with just 27 examples. Extending to 6 letters was easy since ImageFolder handles extra classes automatically.

The experiments showed that more filters consistently helped, pool size barely mattered, and learning rate had the biggest impact on the Greek letter task: from 0.0001 barely learned, while 0.005 hit 100% every time.

## Acknowledgements

AI tools, including Gemini and Claude, were used to set up the VSCode development environment for PyTorch, PyTorch MNIST tutorial. Claude Opus 4.6 also helped us generate parts of the comments, docs, and debug code.

**GitHub repo link:**
https://github.com/ParkerCai/PRCV_Project-5_Recognition-using-Deep-Networks